Contents

Guide 2 — The Recommendation Engine (TOPSIS), the simple version

How Mobile Match Algeria picks the best 3 plans for a user — in plain words, with the research each part comes from, a fully worked example, and ready answers for the jury.

---

1. The idea, in three sentences

The user gives their **budget**, their **data need**, and **ranks** what matters most (price, data). The engine throws away every plan that doesn't fit the hard limits, then ranks the rest by **how close each plan is to the "dream" plan**. It uses no AI and no other users' data, only this user's profile, so it works from day one.

---

2. How it works — five steps

1. **Filter:** drop any plan over budget, or with the wrong operator / type / network.
2. **Weights:** a fixed formula (Rank-Order Centroid) turns the user's ranking into numbers — price #1, data #2 → **0.75 / 0.25**. (We don't pick these; see §7.)
3. **Two reference plans:** the **dream** (best value — lowest price per GB — *and* most data) and the **nightmare** (worst value, least data).
4. **Closeness:** measure how near each plan is to the dream versus the nightmare — a score from **0 (worst) to 1 (best)**.
5. **Rank by closeness**, show the **top 3** (the score becomes the match %).

---

3. The two formulas

Distance between two dots is the **Euclidean (straight-line) distance**, where x is a dot's price-score and y its data-score:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Closeness is how near a plan is to the dream versus the nightmare:

$$C = \frac{d_{\text{nightmare}}}{d_{\text{dream}} + d_{\text{nightmare}}}$$

A plan sitting on the dream gets $C = 1$; one sitting on the nightmare gets $C = 0$.

---

4. Where each part comes from (the research)

Nothing here is invented — each step is a published method:

Part of the engine	What it does	Comes from (research)
Hard filters	remove plans that don't fit	**Conjunctive (constraint) method** — non-compensatory screening (**Hwang & Yoon, 1981**)
Weights from a ranking	"price #1, data #2" → 0.75 / 0.25	**Rank-Order Centroid — Barron & Barrett (1996)**
Ranking by closeness	the formula in §3	**TOPSIS — Hwang & Yoon (1981)**; recent guide: **Taherdoost & Madanchian (2023)**

What is *ours* are setup choices, not formulas: which attributes are filters vs ranked criteria (price, data), that the budget is a hard ceiling, that price is scored as value for money (dinars per GB) rather than raw price, and two small conventions ("unlimited" counts as the best value; equal weights if the user ranks nothing). None of these computes a score — the scoring is entirely the cited methods.

---

5. A worked example — with the full numbers

Setup: **budget 2000 DA**, **need 20 GB**, user ranks **price #1, data #2**. Three plans pass the budget filter:

Plan	Price	Data	Value (price ÷ data)
A	600 DA	30 GB	**20 DA/GB** (best value)
B	2000 DA	50 GB	40 DA/GB (most data)
C	400 DA	5 GB	80 DA/GB (cheapest, worst value)

Price is scored as value for money, not raw dinars. The engine divides each plan's price by its data, so "price" really means **cost per GB**. This is the key idea: ranking price first finds the **best value**, not the cheapest near-empty plan. (That is what fixes the old behaviour where "price first" kept surfacing a tiny 400 DA plan with almost no data.)

Step 1 — turn each plan into two comparable scores. DA/GB and GB can't be compared directly, so we rescale each column. First a **scaling number** for the column = square every value, add them, take the square root:

$$\text{price scaling} = \sqrt{20^2 + 40^2 + 80^2} = \sqrt{8400} = 91.65$$

$$\text{data scaling} = \sqrt{30^2 + 50^2 + 5^2} = \sqrt{3425} = 58.52$$

Then each **score = (value ÷ the column's scaling number) × its weight** (price × 0.75, data × 0.25). Price here is the cost per GB:

Plan	price-score = (DA/GB ÷ 91.65) × 0.75	data-score = (GB ÷ 58.52) × 0.25
A	(20 ÷ 91.65) × 0.75 = **0.164**	(30 ÷ 58.52) × 0.25 = **0.128**
B	(40 ÷ 91.65) × 0.75 = **0.327**	(50 ÷ 58.52) × 0.25 = **0.214**
C	(80 ÷ 91.65) × 0.75 = **0.655**	(5 ÷ 58.52) × 0.25 = **0.021**

Each plan is now **two numbers** — picture it as a dot on a graph (across = price-score, up = data-score).

Step 2 — build the dream and nightmare dots from those two score columns. Price (cost per GB): lower is better, so the *smallest* price-score is best. Data: higher is better, so the *largest* data-score is best:

- **Dream** = smallest price-score **0.164** (A's) + largest data-score **0.214** (B's) → **(0.164, 0.214)**

- **Nightmare** = largest price-score **0.655** (C's) + smallest data-score **0.021** (C's) → **(0.655, 0.021)**

Step 3 — plug each dot into the two formulas from §3. Take **A** (dot $x = 0.164$, $y = 0.128$), measured against the dream $(0.164,\ 0.214)$ and the nightmare $(0.655,\ 0.021)$.

Distance to the dream:

$$d_{\text{dream}} = \sqrt{(0.164 - 0.164)^2 + (0.128 - 0.214)^2} = \sqrt{0 + 0.0074} = 0.086$$

Distance to the nightmare:

$$d_{\text{nightmare}} = \sqrt{(0.164 - 0.655)^2 + (0.128 - 0.021)^2} = \sqrt{0.241 + 0.011} = 0.503$$

Closeness:

$$C = \frac{0.503}{0.086 + 0.503} = \frac{0.503}{0.589} = 0.85$$

The same two formulas, applied to every plan, give:

Plan	distance to dream	distance to nightmare	closeness C	Rank
A	0.086	0.503	0.503 / 0.589 = **0.85**	1st
B	0.163	0.381	0.381 / 0.544 = **0.70**	2nd
C	0.528	0.000	0.000 / 0.528 = **0.00**	3rd

Result: A → B → C. A has the best value for money (20 DA/GB), so on price it sits *exactly on the dream* (distance 0 on the price axis) and price weighs most → highest closeness, 0.85. B has the most data but worse value → 0.70. C is the cheapest in dinars, but its value is the worst *and* it has the least data, so it sits right on the nightmare → 0.00. **Notice the cheapest plan (C) comes last** — "price first" rewards value for money, not a low sticker price. If the user ranked **data first**, the 0.75 weight would shift to data and **B** (the most data) would jump to the top.

---

6. The files

- **recommendation.ts** — the engine: filters, `rocWeights()`, `topsis()`.
- **api/recommendations/route.ts** — the API (validated, rate-limited).
- **recommend/page.tsx** — the form + ranked cards (with savings).

---

7. Jury questions — ready answers

- **What are you calculating? A straight-line (Euclidean) distance?** → Yes. Each plan's distance to the dream and the nightmare plan, combined into a closeness score (TOPSIS).
- **Why doesn't "price first" just give the cheapest plan?** → Because price is scored as **value for money** (dinars per GB), not the raw sticker price. So ranking price first returns the plan that gives the most data per dinar within budget, not a cheap near-empty plan. The budget is still a strict ceiling.
- **Why are the weights 0.75 and 0.25, not 0.70 / 0.30?** → We don't pick them. The **Rank-Order Centroid** formula turns a *ranking* into weights, and for two ranked criteria it always gives exactly 0.75 and 0.25. Choosing 0.70/0.30 would be inventing numbers — ROC avoids that.
- **Did you invent the formula?** → No. Filtering = conjunctive method, weights = ROC, ranking = TOPSIS — all published (Hwang & Yoon 1981; Barron & Barrett 1996). Only the setup choices (which attributes are filters vs criteria, the budget ceiling) are ours.
- **Does it work with no users?** → Yes, it's content-based (plan attributes + the user's profile), so there's no cold-start problem.

- **Why TOPSIS, not AI / a weighted sum / lexicographic / AHP?** → AI needs a usage history we don't have; a weighted sum is unbounded and scale-sensitive; strict lexicographic is too rigid; AHP needs many pairwise comparisons. TOPSIS is bounded, content-based, and pairs naturally with the ranking-based weights.
- **What if no plan fits?** → The list is empty (the budget ceiling is strict); the user can raise the budget or relax a filter.

Next guide: the search engine (token parser + FTS5). Tell me when you want it.